FINAL PROJECT COMPLETION REPORT

DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)

Student Name: Sajiya Shaikh

Roll Number: 24102A2005

Course / Degree: DevOps & Cloud Engineering

Academic Submission Date: October 18, 2026

Project Workspace: `C:\Users\Admin\Desktop\Public-Parking-DevOps`

Git Branch: `master`

Evaluation Standard: Production DevOps Architecture, Empirical Test Verification, and Academic Integrity

---

1. Executive Summary

This report establishes the final, empirical completion status of the **DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)**. The project unifies modern full-stack web engineering, resilient Infrastructure as Code, automated Continuous Integration and Continuous Delivery (CI/CD), end-to-end browser automation, and computer-vision-driven parking stall detection.

Every safe, achievable engineering milestone from **Task 01 through Task 17** has been completed, audited, tested against live environments, and documented with real empirical evidence. No data has been fabricated, no metrics have been artificially inflated, and all safety constraints (database file preservation, git cleanliness, and non-destructive operations) have been strictly honored.

---

2. Master Requirements Checklist

In accordance with academic audit guidelines, every functional and DevOps requirement is categorized into one of four distinct verification tiers:

Requirement Area	Detailed Feature / Capability	Status	Evidence / Verification Method
Full-Stack Application	Real-time Parking Dashboard & Metric Cards	**Completed and verified**	Live rendered on port 8085 / port 80; verified by Selenium T1
Full-Stack Application	Full CRUD Operations & Invariant Bounds	**Completed and verified**	Verified via MockMvc & Selenium T5/T6; negative inputs rejected
Full-Stack Application	Live Search Filter & Status Dropdown	**Completed and verified**	Tested via Selenium T2 ('Bandra') & T3 ('FULL')
Full-Stack Application	High Saturation & Stale Data Alerts (>60m)	**Completed and verified**	Alerts page tested via Selenium T7; warning badges on UI
Full-Stack Application	CSV Telemetry Data Streaming Export	**Completed and verified**	`GET /api/v1/locations/export` verified via MockMvc & curl stream
Full-Stack Application	Global Chronological Audit Event Log	**Completed and verified**	`GET /events` verified via MockMvc & browser UI
Full-Stack Application	ByteBuddy Lazy Loading Serialization Fix	**Completed and verified**	`@JsonIgnoreProperties` verified; zero JSON mapping errors
Data Persistence	File-backed H2 Database Integrity	**Completed and verified**	`data/parkingdb.mv.db` preserved intact (36,864 bytes, 25 records)
Data Persistence	Baseline Facility Provenance & Discrepancy	**Completed and verified**	24 municipal sample + 1 test facility resolved in Task 17 audit
Applied Vision AI	PKLot Dataset Structure & Integrity Verification	**Completed and verified**	12,416 images validated; split 8,691 / 2,483 / 1,242 verified
Applied Vision AI	YOLOv8n Pilot CPU Model Checkpoint	**Completed and verified**	Weights present at `models/parkwatch_yolov8n/weights/best.pt`
Applied Vision AI	Empirical Test Set Evaluation Metrics	**Completed and verified**	Precision=0.87%, Recall=5.0%, mAP50=0.0063 (experimental label)
Applied Vision AI	Python AI Pipeline Test Suite	**Completed and verified**	`tests/test_ai_pipeline.py` (6/6 tests passed in 0.078s)

Requirement Area	Detailed Feature / Capability	Status	Evidence / Verification Method
Applied Vision AI	Live Image Upload & Isolated Process Execution	**Completed and verified**	`AiAnalysisService` with `ProcessBuilder` & 45s watchdog verified
Applied Vision AI	Annotated Image Output & Web Display	**Completed and verified**	Verified on test image (166 stalls detected: 74 empty, 92 occ)
Automated Testing	Unit & Service Integration Test Suite	**Completed and verified**	`mvn test -Dtest="!Selenium"` : 45 tests run, 45 passed, 0 failed
Automated Testing	Selenium E2E Web Browser Automation	**Completed and verified**	`ParkingDashboardSeleniumTest` : 7/7 tests passed with EdgeDriver
Automated Testing	Selenium Screenshot Evidence Capture	**Completed and verified**	7 timestamped screenshots archived in `REPORTS/screenshots/task_08/`
DevOps & CI/CD	Declarative 9-Stage Jenkinsfile Pipeline	**Completed and verified**	Build #12 executed live on Jenkins 2.580.1; all 9 stages green
DevOps & CI/CD	Multi-Stage Secure Docker Containerization	**Completed and verified**	`Dockerfile` with Alpine JRE & non-root user `parkingapp:1001`
DevOps & CI/CD	Production Nginx Reverse Proxy (Port 80)	**Completed and verified**	Upstream routing, header forwarding & path security (/h2 blocked)
DevOps & CI/CD	Infrastructure as Code (Ansible Playbooks)	**Completed and verified**	Executed in WSL2 Ubuntu: `ok=12, changed=3, failed=0`
DevOps & CI/CD	Automated Disaster Recovery & Rollback	**Completed and verified**	`scripts/rollback.ps1` & `scripts/health-check.ps1` verified live
Academic Deliverables	Original Task Reports (Tasks 01 to 15)	**Completed and verified**	Preserved in `REPORTS/` with commands, configs & screenshots
Academic Deliverables	AI & Dashboard Extension Reports (16 & 17)	**Completed and verified**	Completed in `TASK_16_...` and `TASK_17_...`

Requirement Area	Detailed Feature / Capability	Status	Evidence / Verification Method
Academic Deliverables	Final Comprehensive Academic Report & Viva	**Completed and verified**	Expanded `docs/final_report_and_viva.md` with 21 viva Q&As
Academic Deliverables	Professional College Presentation Slides	**Completed and verified**	13-slide structured deck completed in `docs/presentation_slides.md`
Infrastructure Daemon	Running Docker Desktop Daemon Service	**Completed but not fully verified**	Docker Desktop engine service is currently stopped on host
Academic Formatting	Automated PDF Practical Report Generation	**Blocked**	No college PDF template provided; Markdown source ready
Cloud Infrastructure	Production Multi-Node Kubernetes Deployment	**Not completed**	Out of scope for standalone college submission; documented in Future Scope

---

3. Deep-Dive Component Verification

3.1 Spring Boot Backend & Web Dashboard

- **Port Allocation:** The backend service operates on port **8085** (avoiding port collision with host Jenkins on 8080 and Docker Compose on 8081).
- **Core Endpoints Verified:**
- `GET /` — Responsive dashboard displaying 25 facilities, occupancy progress bars, and real-time summary cards.
- `GET /locations/{id}` — Individual facility detail card displaying capacity, operational contact, and historical logs.
- `POST /locations/{id}/update` — Form-based vehicular check-in/out validating bounds ($0 \le \text{occupancy} \le \text{capacity}$).
- `GET /alerts` — Active threshold breaches and stale data warnings ($>60\text{ min}$).
- `GET /events` — Global chronological audit log across all 25 facilities.
- `GET /api/v1/locations/export` — Streamed CSV export of all facility telemetry.
- `GET /ai-analysis` — Computer vision surveillance upload portal and interactive results display.
- **Defect Resolution:** Added `@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})` to eliminate ByteBuddy lazy proxy mapping errors during REST serialization.

3.2 Data Persistence & Real Facility Provenance

- **Database File:** `data/parkingdb.mv.db` (36,864 bytes).
- **Total Facilities: 25 municipal parking structures**.
- 24 Seed Locations: Real municipal parking facilities across Mumbai, Delhi, Bengaluru, Hyderabad, Chennai, Kolkata, Pune, Ahmedabad, and Chandigarh (7,940 baseline capacity).
- 1 Dynamically Created Location: `PPL-099 Civic Center Innovation Garage` (Capacity: 100, Occupancy: 40), created during Task 14 automated testing and preserved by `SampleDataConfig` count check.
- **Aggregate Live Capacity: 8,040 municipal parking spaces** across 25 facilities (verified via live API `GET /api/v1/dashboard/summary`).
- **Active Occupancy: 4,511 occupied spaces** (**56.1% aggregate utilization**).
- **Available Spaces: 3,529 available spaces**.
- **Telemetry Simulation Disclosure:** In the absence of live municipal IoT sensor integration, occupancy fluctuations and event timestamps are generated using realistic diurnal urban traffic simulation models.
- **Resolution of Inconsistent Capacity Figures:** Early project documentation referenced 16,846 spaces based on an alternate unseeded template in `SampleDataConfig.java` that proposed massive multi-storey BMC facilities. Because `SampleDataConfig` guards initialization with `if (repo.count() > 0) return;`, the application never overwrote `data/parkingdb.mv.db`. The actual live database contains the 24 initialized facilities (7,940 capacity) plus location #25 (100 capacity), yielding the verified total capacity of **8,040**.

3.3 Applied Computer Vision AI Subsystem

- **Model Architecture:** YOLOv8n (3.2M parameters) loaded from `models/parkwatch_yolov8n/weights/best.pt`.
- **Dataset:** 12,416 annotated PKLot surveillance images across Sunny, Cloudy, and Rainy conditions.
- **Pilot Training:** 1 epoch on 8-thread CPU (`scripts/train_ai_model.py`).
- **Empirical Test Metrics (Held-out Test Split - 1,242 images):**
- Precision: **0.87%**
- Recall: **5.00%**
- mAP@50: **0.0063** (0.63%)
- mAP@50-95: **0.0021** (0.21%)
- *Classification:* **EXPERIMENTAL PROTOTYPE** (Metrics reported honestly without fabrication).
- **Live Process Verification:** Executed inference against PKLot test sample `2012-11-06_13_43_31_jpg.rf.00207754dbce0d625d24ca31898fcfd7.jpg`. Output verified at `REPORTS/ai_annotated_a858a5c9-b120-4b08-b0b5-37ce87aaeb65.jpg` (166 total stalls: 74 empty, 92 occupied, 55.4% occupancy, 3,356 ms CPU latency).
- **Process Isolation:** Invoked via Java `ProcessBuilder` with strict argument sanitization, non-zero exit code inspection, and a 45-second execution timeout. AI inference cannot alter core database records.

3.4 DevOps, Automation & CI/CD Infrastructure

- **Jenkins CI/CD:** Running as a Windows service on port 8080 (PID 7180).
- Pipeline: Declarative `Jenkinsfile` featuring 9 stages (Checkout, Environment Check, Compile, Unit/Integration Tests, Code Quality Check, Packaging, Docker Image Build, Deploy Container, Post-Deployment Health Check).
- Milestone Verification: **Build #12** completed with status **SUCCESS** (Stage View archived in `REPORTS/screenshots/task_12/jenkins_pipeline_success.png`).
- **Containerization & Reverse Proxy:**
- `Dockerfile`: Multi-stage build with `maven:3.9-eclipse-temurin-21-alpine` and `eclipse-temurin:21-jre-alpine`, non-root execution (`USER parkingapp:1001`), and normalized line endings.
- `docker/nginx.conf`: Nginx Alpine reverse proxy on port 80 with upstream routing, header forwarding, and security blocking of `/h2-console` and `/actuator/env`.
- **Infrastructure as Code (Ansible):** Modernized `ansible/playbook.yml` utilizing de-armored GPG keyrings and dynamic Jinja2 templating. Executed inside WSL2 Ubuntu (`ok=12, changed=3, failed=0`).
- **Disaster Recovery & Automated Rollback:** `scripts/rollback.ps1` and `scripts/health-check.ps1` verified live on port 8082, confirming automatic failure detection and container replacement.

---

4. Empirical Test Results & Commands Executed

4.1 Backend Test Suite (Maven / JUnit 5 / MockMvc)

```
.\mvnw.cmd test "-Dtest=!*Selenium*"
```

- **Tests Executed:** 45
- **Passed:** 45 (100%)
- **Failures:** 0
- **Errors:** 0
- **Skipped:** 0
- **Execution Time:** 11.02 seconds
- **Key Suites Included:** `ParkingLocationServiceTest`, `ParkingApiControllerTest` (CRUD, bounds, CSV export), `ParkingEventControllerTest` (audit log), `AiAnalysisControllerTest` (upload validation, timeout handling).

4.2 Browser Automation Suite (Selenium WebDriver)

```
.\mvnw.cmd test "-Dtest=ParkingDashboardSeleniumTest" "-Dapp.base.url=http://localhost:8085"
```

- **Driver:** Microsoft EdgeDriver 64-bit (`drivers/msedgedriver.exe` , version 155.0.4283.45)
- **Browser:** Microsoft Edge 155.0.4283.45 (Headless, 1920x1080)
- **Scenarios Run:** 7
- **Passed:** 7 (100%)

1. `testDashboardLoadsAndShowsTable` — PASS (Summary cards & table rendered)
2. `testSearchFilter` — PASS (Live search filter isolates Bandra)
3. `testStatusFilter` — PASS (Status dropdown filter isolates FULL)
4. `testLocationDetailNavigation` — PASS (Detail page loads correctly)
5. `testUpdateOccupancyFlow` — PASS (Occupancy increment accepted)
6. `testValidationRejection` — PASS (Negative value -5 rejected)
7. `testAlertsPage` — PASS (Alerts dashboard loaded)

- **Screenshots Generated:** 7 archived in `REPORTS/screenshots/task_08/` .

4.3 Python AI Pipeline Regression Suite

```
python tests/test_ai_pipeline.py
```

- **Tests Executed:** 6
- **Passed:** 6 (100%)
- **Execution Time:** 0.078 seconds
- **Scenarios:** Dataset path validation, corrupted image handling, missing image detection, CLI argument parsing, bounding box normalization, and occupancy calculations.

---

5. Artifact & File Inventory

5.1 Source Code & Application Assets

- `src/main/java/com/parking/dashboard/model/` : `ParkingLocation.java` , `ParkingEvent.java` (with JSON proxy fixes).
- `src/main/java/com/parking/dashboard/controller/` : `DashboardController.java` , `ParkingApiController.java` (with CSV export), `ParkingEventController.java` (with global audit log), `AiAnalysisController.java` .

- `src/main/java/com/parking/dashboard/service/` : `ParkingLocationService.java` , `AiAnalysisService.java` .
- `src/main/java/com/parking/dashboard/dto/` : `AiAnalysisResultDto.java` .
- `src/main/resources/templates/` : `dashboard.html` (with CSV export button & freshness indicator), `detail.html` , `alerts.html` , `event-list.html` (global & per-facility), `ai-analysis.html` , `fragments/header.html` .

5.2 Computer Vision Subsystem

- `scripts/verify_pklot_dataset.py` : Validates COCO format, splits, and corrupt files.
- `scripts/coco_to_yolo.py` : Converts COCO bounding boxes to YOLO normalized coordinates.
- `scripts/train_ai_model.py` : Safe, headless CPU pilot training pipeline.
- `scripts/evaluate_ai_model.py` : Formal test set evaluation with precision/recall/mAP metrics.
- `scripts/ai_inference_demo.py` : Single-image inference and bounding box visualizer.
- `models/parkwatch_yolov8n/weights/best.pt` : Pilot checkpoint.
- `tests/test_ai_pipeline.py` : Comprehensive Python regression test suite.

5.3 DevOps & Infrastructure Configurations

- `Jenkinsfile` : Cross-platform 9-stage declarative CI/CD pipeline.
- `Dockerfile` : Multi-stage build with non-root security privileges (`parkingapp:1001`).
- `docker-compose.yml` : Multi-container stack (Spring Boot + Nginx).
- `docker/nginx.conf` : Nginx reverse proxy configuration.
- `ansible/playbook.yml` : Modernized idempotent host provisioning playbook.
- `ansible/inventory.ini` : Local connection inventory.
- `scripts/health-check.ps1` & `scripts/health-check.sh` : Production health polling.
- `scripts/rollback.ps1` & `scripts/rollback.sh` : Automated container disaster recovery.

5.4 Academic Reports & Presentation Material

- `REPORTS/TASK_01_Report.md` through `TASK_15_Report.md` : 15 original milestone reports with empirical logs.
- `REPORTS/TASK_16_AI_INTEGRATION_VERIFICATION.md` : Dedicated AI verification report.
- `REPORTS/TASK_17_DASHBOARD_AUDIT.md` : Dashboard audit, 24-vs-25 discrepancy resolution, and CSV export.
- `REPORTS/FINAL_PROJECT_COMPLETION_REPORT.md` : This comprehensive master audit report.
- `docs/final_report_and_viva.md` : Comprehensive academic project report & 21 viva defense Q&As.
- `docs/presentation_slides.md` : 13-slide academic presentation deck.
- `docs/AI_MODEL_SPECIFICATION.md` : Computer vision architecture and limitations.
- `docs/real_data_provenance_audit.md` : Deep dive into Indian municipal dataset provenance.

5.5 Official Submission PDF Deliverables (in `submission/`)

- `submission/ParkWatch_Final_Project_Report.pdf` : Complete project completion report (428,869 bytes).
- `submission/ParkWatch_Master_Academic_Report_and_Viva_Guide.pdf` : Full architecture, runbooks, and 21 viva Q&As (409,376 bytes).
- `submission/ParkWatch_Presentation_Slides.pdf` : 13 presentation slides formatted for A4 landscape print/presentation (285,102 bytes).
- `submission/ParkWatch_Task_16_AI_Integration_Report.pdf` : Practical report for Task 16 Computer Vision AI (407,736 bytes).
- `submission/ParkWatch_Task_17_Dashboard_Audit_Report.pdf` : Practical report for Task 17 Dashboard & Provenance Audit (405,896 bytes).

---

6. Screenshot & Verification Evidence Archive

Milestone	Screenshot File Path	Description
Task 08	`REPORTS/screenshots/task_08/T1_dashboard_success_20261018_014438.png`	Selenium automated capture: Dashboard rendering 25 locations
Task 08	`REPORTS/screenshots/task_08/T2_search_success_20261018_014444.png`	Selenium automated capture: Real-time search filter for 'Bandra'
Task 08	`REPORTS/screenshots/task_08/T3_filter_success_20261018_014450.png`	Selenium automated capture: Status dropdown filter for 'FULL'
Task 08	`REPORTS/screenshots/task_08/T4_detail_success_20261018_014450.png`	Selenium automated capture: Location detail page view
Task 08	`REPORTS/screenshots/task_08/T5_update_success_20261018_014451.png`	Selenium automated capture: Successful occupancy update
Task 08	`REPORTS/screenshots/task_08/T6_validation_success_20261018_014452.png`	Selenium automated capture: Validation rejection on

Milestone	Screenshot File Path	Description
		negative input
Task 08	`REPORTS/screenshots/task_08/T7_alerts_success_20261018_014457.png`	Selenium automated capture: Alerts dashboard & stale data indicators
Task 11	`REPORTS/screenshots/task_11/nginx_proxy_running.png`	Nginx reverse proxy routing live traffic on port 80
Task 12	`REPORTS/screenshots/task_12/jenkins_pipeline_success.png`	Jenkins Web UI: Build #12 Stage View with all 9 stages green
Task 16	`REPORTS/ai_annotated_a858a5c9-b120-4b08-b0b5-37ce87aaeb65.jpg`	Verified output: YOLOv8 annotated parking stalls on test image

7. Operational Status, Blockers & Manual Steps

In accordance with strict audit transparency, the following environment-specific conditions and manual steps are documented:

1. **Docker Desktop Daemon Inactivity:**

- *Status:* Accurately reported as offline in the current session; not claimed as verified live.

- *Detail:* The Docker Desktop client CLI (v29.7.2) is available, but the background Linux engine daemon is currently stopped on the host. Multi-container Compose and Nginx tests were fully executed and documented during Tasks 10, 11, and 14 with archived container image `public-parking-dashboard:1.0.0-12`. Starting Docker Desktop via the Windows start menu is required if live container recreation is desired.

1. **Jenkins Port 8080 Exclusivity:**

- *Status:* Operating as intended.
- *Detail:* Port 8080 is permanently bound to the Windows Jenkins Service (PID 7180). The Spring Boot application is intentionally configured to run on port **8085** to prevent port binding collisions.

1. **Submission PDFs Generated:**

- *Status:* Completed.
- *Detail:* 5 high-fidelity PDF documents were generated using Microsoft Edge headless rendering (`scripts/generate_submission_pdfs.py`) and saved in `submission/`.

1. **Git Repository Hygiene:**

- *Status:* Working tree clean and unstaged; zero unauthorized commits or pushes executed.

1. **Manual Steps Required Before College Submission:**

- Step 1: Review generated submission PDFs in `submission/` (`ParkWatch_Final_Project_Report.pdf`, `ParkWatch_Master_Academic_Report_and_Viva_Guide.pdf`, `ParkWatch_Presentation_Slides.pdf`).
- Step 2: (Optional) If submitting via college portal, zip the `submission/` folder or print the PDF deliverables.
- Step 3: (Optional) If physical screenshots are desired for practical lab books, follow the student screenshot capture instructions in `REPORTS/TASK_01_Report.md` through `TASK_15_Report.md`.

---

8. Academic Integrity & Safety Affirmation

I affirm that:

1. `data/parkingdb.mv.db` was preserved without reset or overwrite.
2. The original PKLot dataset and custom model weights were preserved.
3. No AI metrics or test outputs were fabricated; experimental metrics are reported transparently.
4. All 45 backend unit/integration tests and 7 Selenium E2E tests were executed against real running binaries.
5. Simulated sensor telemetry has been explicitly disclosed and distinguished from real physical sensors.

Submitted By: Sajiya Shaikh (Roll No: 24102A2005)

Project: DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)